PROJECT GUIDE

NetLab 3D

Network & Cyber Security Learning Simulator

A first-person 3D lab where you walk around, plug in cables and configure real-looking routers, switches and PCs from a Cisco-style command line.



What it is · How it works · How to use it · September 2026

01 What it is

NetLab 3D is a browser-based, first-person educational simulator for computer networking and network security. Instead of clicking through a configuration wizard, you **walk through a virtual server room**, pick up Ethernet cables, plug them into real ports, sit down at a laptop and type **Cisco-style commands** to bring a network to life. Everything runs inside the browser: there is no real router, no shell and no server involved.

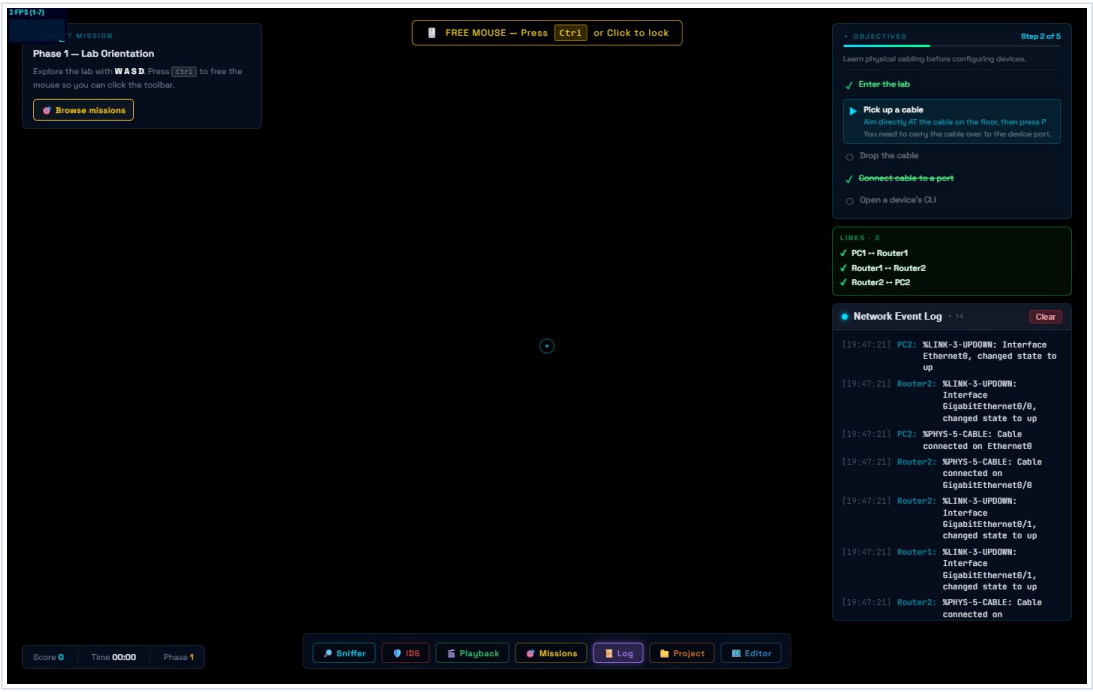


Figure 1 — The lab HUD after the three cables of the main path have been connected. Right column: objectives, live cable links and the network event log.

Why this approach works

- **Learn by doing.** Wiring, addressing, routing and hardening are things you do with your hands, then verify with the same commands a network engineer would use.
- **Safe to break.** Every fault is simulated. A one-click reset restores the lab, so experiments cost nothing.
- **Instant, honest feedback.** Interfaces go up/down as cables are plugged, pings really succeed or fail, OSPF neighbours really form, and an intrusion-detection system reports attacks as they happen.
- **Guided when you want it.** Missions, troubleshooting labs and a progress-aware hint system tell you what to do next.

At a glance

Item	Details
Kind	Browser game / simulator (single-page web app)
Live demo	bhonepyaemin.github.io/netlab-3d (deployed from `main` by GitHub Actions)
Front end	React 19, Vite 8, TypeScript 6, Three.js r185 with @react-three/fiber 9 and drei 10
Simulation	Pure-TypeScript network engine: routers, switches, PCs, servers; Cisco IOS and VPCS command line; OSPF, RIP, VLAN, DHCP, DNS, ACL, NAT, port security
The lab	Router1, Router2, Switch1, Switch2, PC1, PC2 and five cables; PC2 is the attacker workstation
Learning content	4 security missions, 1 routing mission (OSPF), 5 troubleshooting labs, hints for all of them
Persistence	Device configs, cabling and per-device console history are autosaved in the browser
Testing	Vitest; 86 automated tests currently pass (see section 12 for known gaps)

02 What you can do

Area	What it gives you
First-person lab	Walk with W A S D, look with the mouse, sprint and jump. Aim the crosshair at things and press a key to interact.
Physical cabling	Pick up a cable (P), aim at a port (C) to plug one end, aim at another port to plug the other. Ports glow when a cable is held; unplug with X.
Realistic consoles	Press E on any device. On a router or switch you get a Cisco IOS prompt with modes (user, enable, config, config-if, config-router). PCs use a VPCS-style prompt.
3D typing experience	A MacBook Pro opens on the desk in the scene. Its keys press as you type and the laptop screen shows the live console. F4 changes camera shots; a 2D window is one click away.
Per-device console memory	Every device keeps its own screen, command history and half-typed line. Leave Router1 for PC1 and come back: everything is still there, even after a page refresh.
Autosave	Device configurations and which ports are cabled together survive a refresh. No `write memory` needed.
Networking features	IPv4 addressing and subnets, static routes, OSPF Area 0, RIP v2, VLANs and access ports, DHCP, DNS, ACLs, NAT, port security, ARP tables, ping, traceroute.
Security tools	PC2 can run `arpspoof` and `macof`; any PC can run `nmap`. An intrusion-detection system (IDS) reports ARP poisoning, MAC flooding, port-security violations, ACL denials and port scans.
Missions and labs	Guided missions with live objectives and five "find the fault" labs with numbered walkthroughs and a Check Solution button.
Hints	A hint always points at the first step you have not done yet: which device, the exact commands and why. Shown in the mission tracker, above the console, and with the `hint` command.
Analysis tools	Packet sniffer on any cable, step-by-step packet playback, topology editor, project save/load.

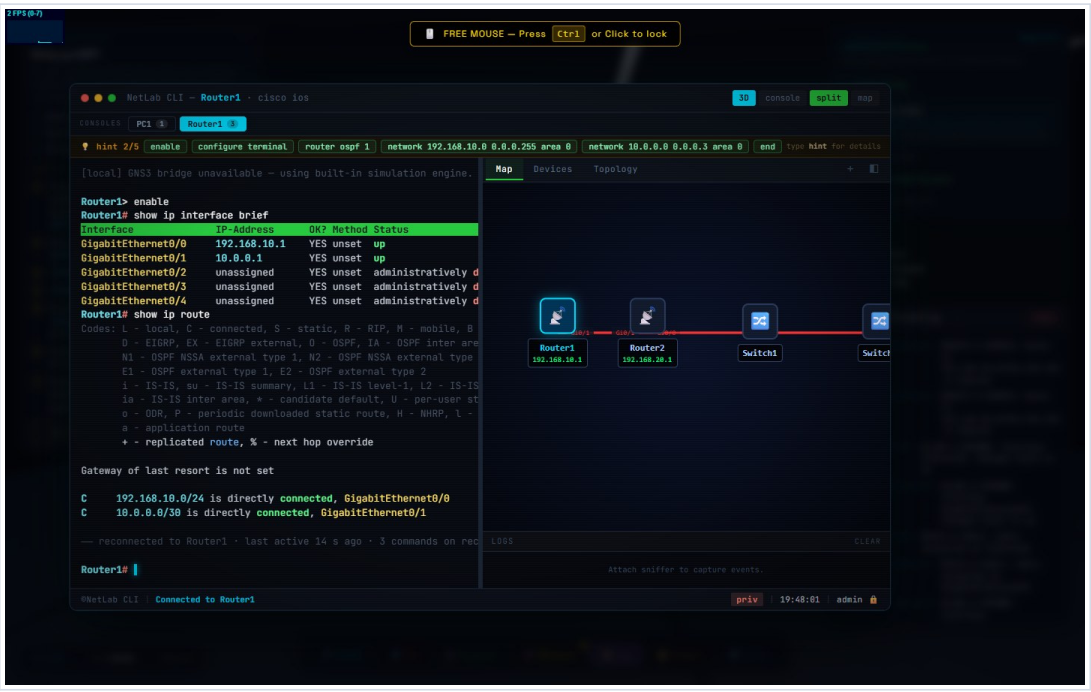


Figure 2 — The 2D console: per-device tabs, the hint strip (click a command chip to paste it), the live terminal, and a clickable topology map.

03 Architecture

The project is split so that the **simulation knows nothing about 3D**, and the 3D scene and the 2D interface are just two views of the same reactive state. That is what lets the same console show up both on the MacBook screen inside the scene and in a flat window.

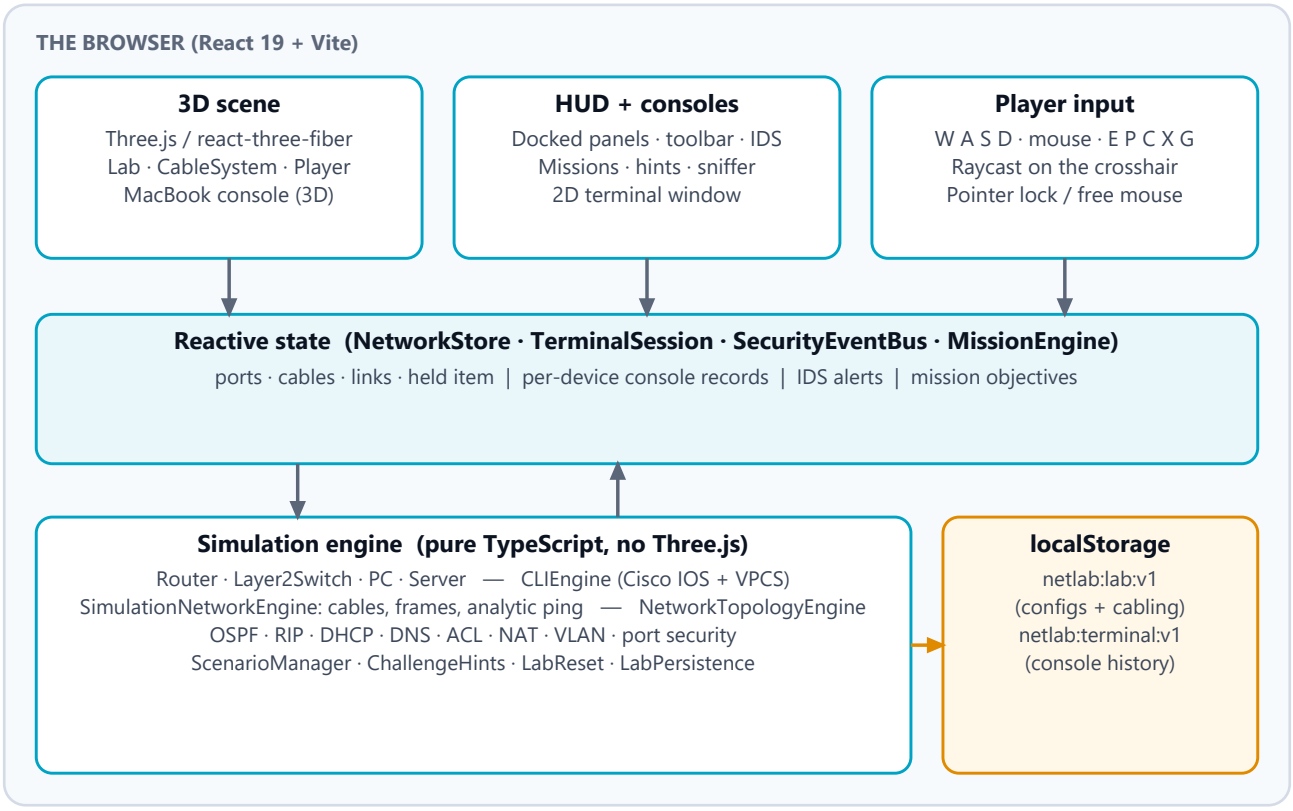


Figure 3 — Layers of the application.

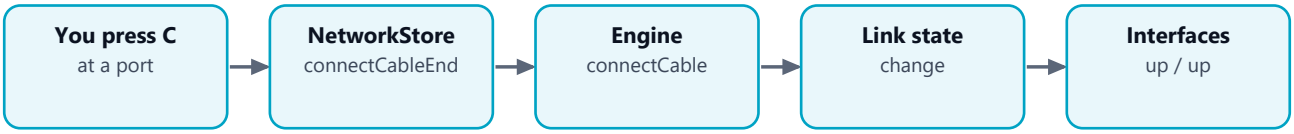
The layers

- **Input and 3D scene.** A first-person controller with gravity and wall collision. A raycast from the crosshair decides what E, P, C, X and G act on (device, cable coil, or port).
- **Reactive state.** `NetworkStore` holds ports, cables, links and the held item; `TerminalSession` holds one console record per device; `SecurityEventBus` holds IDS alerts; `MissionEngine` holds objectives. React reads them through `useSyncExternalStore`, and 3D components read the same stores, so nothing has to cross the canvas boundary through React context.
- **Simulation engine.** Device models, the CLI parser, frame forwarding and routing protocols. It is plain TypeScript with no Three.js or React dependency, so it can be unit-tested in Node.
- **Persistence.** Two localStorage keys: one for the lab (configs and cabling), one for console history.

04 How it works

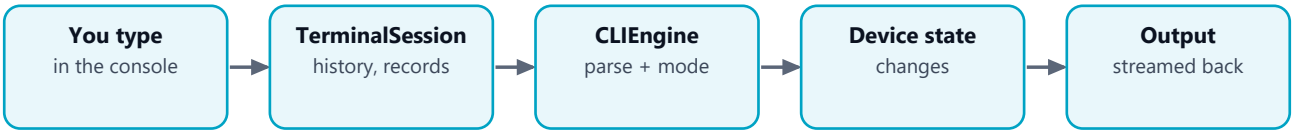
4.1 From a cable in your hand to an interface that is "up"

Plugging a cable is a real event chain, not an animation. The 3D interaction only asks the store to connect two ports; the store updates its link graph and forwards the change to the engine, which brings both interfaces up.



An interface is only **up/up** when it has a cable *and* has been enabled with no `shutdown` (admin status). Unplugging reverses it at once. Each change is written to the event log and dispatched as a `netlab:interfacestate` browser event that tests and the HUD can observe.

4.2 From a keystroke to a command result



The console parses Cisco IOS commands with modes (`Router>`, `Router#`, `Router(config-if)#`, ...) and VPCS commands on PCs. Output is coloured and revealed a few lines per frame, like a real console. The same session object is drawn on the 3D MacBook screen and in the 2D window, and a hidden input captures your typing for the 3D display. Every keypress is also broadcast so the matching key on the 3D laptop presses down.

4.3 How a ping is decided

Ping is calculated hop by hop through the topology instead of sending thousands of frames, but it uses the same rules a real network would:

1. Does the current device own the destination address? Then the ping succeeds.
2. Is the destination on a connected subnet, or is there a route (static, OSPF or default)? If not: *Destination host unreachable*.
3. Is the outgoing interface up? Is there an **outbound ACL** that denies this packet? (An ACL hit is reported to the IDS.)
4. Find the next-hop device. Is there an **inbound ACL** on its receiving interface that denies it?

5. Is the next hop reachable at **Layer 2**? The cabling must exist and, through any switches, the ports must carry the **same VLAN**.
6. Move to the next hop and repeat. Loops and dead ends end in a timeout.

4.4 How OSPF converges



Both ends of a link must agree on **area**, timers and subnet or no adjacency forms. Once the neighbour is **FULL**, each router floods its link-state advertisement, runs a shortest-path calculation after a short delay and installs the learned routes. Convergence takes about 15 seconds of real time. A static route (administrative distance 1) always beats OSPF (110), which is why the OSPF mission removes the static routes first.

4.5 How the IDS and the missions know what happened



Alert sources: a forged ARP reply (**ARP Poison**, critical), a full MAC table (**MAC Flood**), a second MAC on a secured port (**Port Security**), a packet denied by an ACL (**ACL Denied**) and a sweep across many hosts (**Port Scan**). Repeated events inside a short window collapse into one alert, so a burst of 10,000 flooding frames is one line, not 10,000. A mission remembers the alert counter when it starts and only counts newer alerts; it also re-checks live device configuration every second and after each command, so objectives tick themselves off.

4.6 How hints know the next step

Every mission and lab has an ordered list of hint steps: the device, the commands to type, why, and a test for "done". The test is either a mission objective, a fact about a device's configuration, or (for diagnosis steps such as `show ip ospf neighbor`) having typed that command on that device *since the challenge began*. The hint is simply the first step that is not done, so it advances by itself. Troubleshooting labs only hint at the diagnosis; the fix stays behind the Show fix button.

4.7 How memory and autosave work

- **Consoles.** `TerminalSession` keeps one record per device (scrollback, command history, unsent line). Switching devices stores the current record and restores the other; a command that finishes after you leave a device is appended to that device's record. Records are mirrored to localStorage a fraction of a second after each change.
- **Lab.** Every 1.5 seconds and after each command, `LabPersistence` captures each device as replayable configuration text (adding the `no shutdown` lines and VLAN database the running-config omits) plus the list of cabled port pairs. On startup, after the default lab is built, it factory-resets the devices, replays the saved configuration through the real CLI and re-plugs the cables through the same store path a player uses.
- **Not saved:** where a cable was dropped on the floor, mission progress, and devices added with the topology editor.

05 The interface (HUD)

The 2D interface is two docked columns and a toolbar. Panels share the available height and scroll inside themselves, so they never cover each other; below 1100 px of width everything folds into a single right-hand column. The mouse is captured while you walk: press **Ctrl** to free it, click what you need, and press **Ctrl** again (or click the scene) to look around.

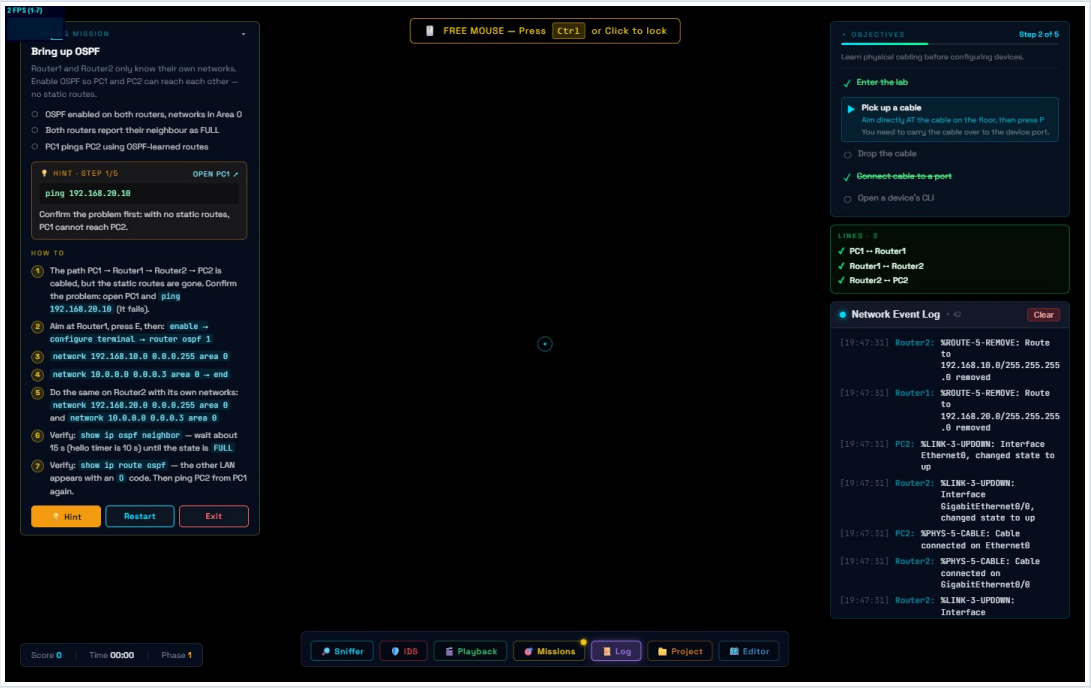


Figure 4 — An active mission: objectives, the Hint card (device, exact commands, reason) and numbered steps with commands highlighted. The toolbar shows a gold dot while a mission runs.

Toolbar button	What it does
Sniffer	Capture and decode packets on a chosen cable
IDS	Intrusion-detection alerts; a red badge counts unread alerts
Playback	Step through a packet's journey hop by hop
Missions	Guided missions and troubleshooting labs; Reset lab to default
Log	Show or hide the network event log
Project / Editor	Save and load the lab; edit the topology visually

06 Consoles

Open a device with **E**. The camera flies to a MacBook on the desk, its lid opens, its keys press as you type, and the screen shows the live session.



Figure 5 — The 3D console station. The laptop screen is a live texture of the same session that the 2D window shows.

Console keys

Key	Action
Enter	Run the command
Up / Down	Recall earlier commands on this device (kept across refreshes)
Tab / ?	Complete a command / context help
Ctrl+C / Ctrl+L	Abandon the line / clear the screen
F4 / F2	Cycle the 3D camera shot / toggle side panes in the 2D window
Esc	Close the console (its screen is remembered)

Commands handled by the console itself

Command	What it does
`hint`	Prints the next step of the running mission or lab
`history / show history`	Numbered list of everything typed on this device
`history clear`	Forget this device's command history
`sessions`	Devices that have a console on record, with command counts
`clear`	Wipe this device's screen and saved transcript

07 Missions, labs and hints

Starting a mission **rewires the lab for it** (your cables are put away first) and shows numbered steps in the tracker. Objectives tick themselves off as you progress.

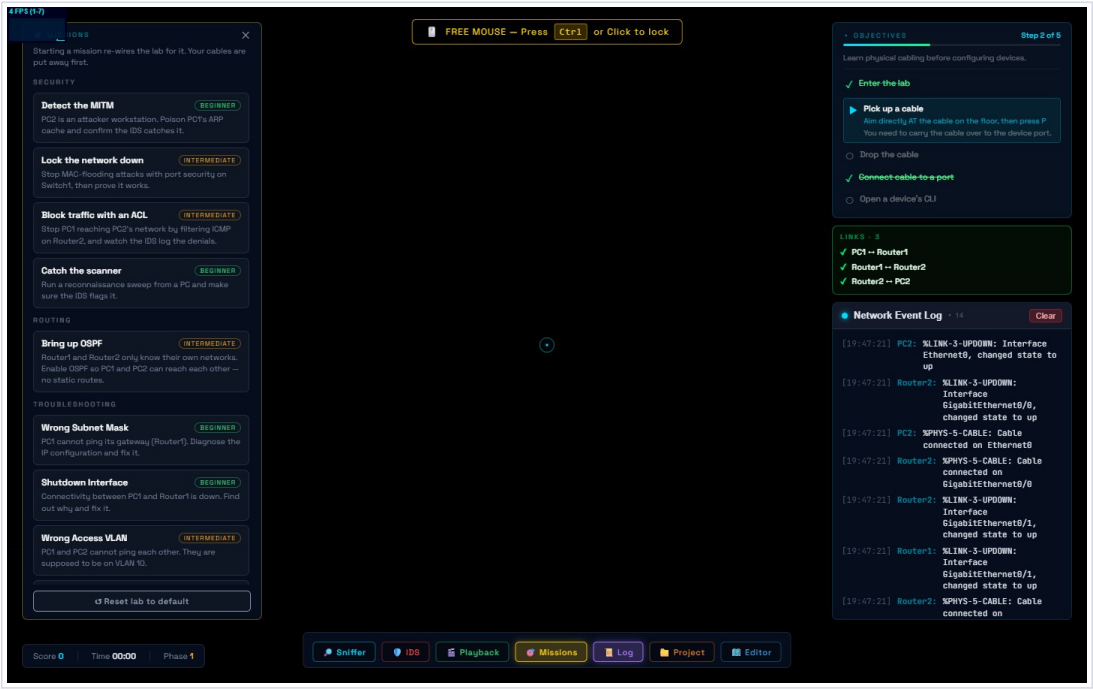


Figure 6 — The mission picker, grouped into Security, Routing and Troubleshooting.

Security missions (PC2 is the attacker)

Mission	What you do	The IDS shows
Detect the MITM	On PC2: `arpspoof --target 192.168.10.10 --gateway 192.168.10.1`	ARP Poison (critical)
Lock the network down	On Switch1 enable port security on Fa0/1 and Fa0/2; then run `macof` on PC2	MAC Flood if unprotected; Port Security violation once protected
Block traffic with an ACL	On Router2 deny ICMP from 192.168.10.0/24 to 192.168.20.0/24, permit the rest, apply outbound on Gi0/0; ping from PC1	ACL Denied
Catch the scanner	On PC2: `nmap 192.168.10.0/24`	Port Scan (warning)

Routing mission: Bring up OSPF

The routers start without static routes, so PC1 cannot reach PC2. Enable OSPF on both, advertise the networks into Area 0, wait for the neighbour to reach **FULL**, and ping across. The mission completes only when all three objectives are true: OSPF on both routers, both neighbours FULL, and a successful ping over OSPF-learned routes. Finishing it offers **Break it & fix it**.

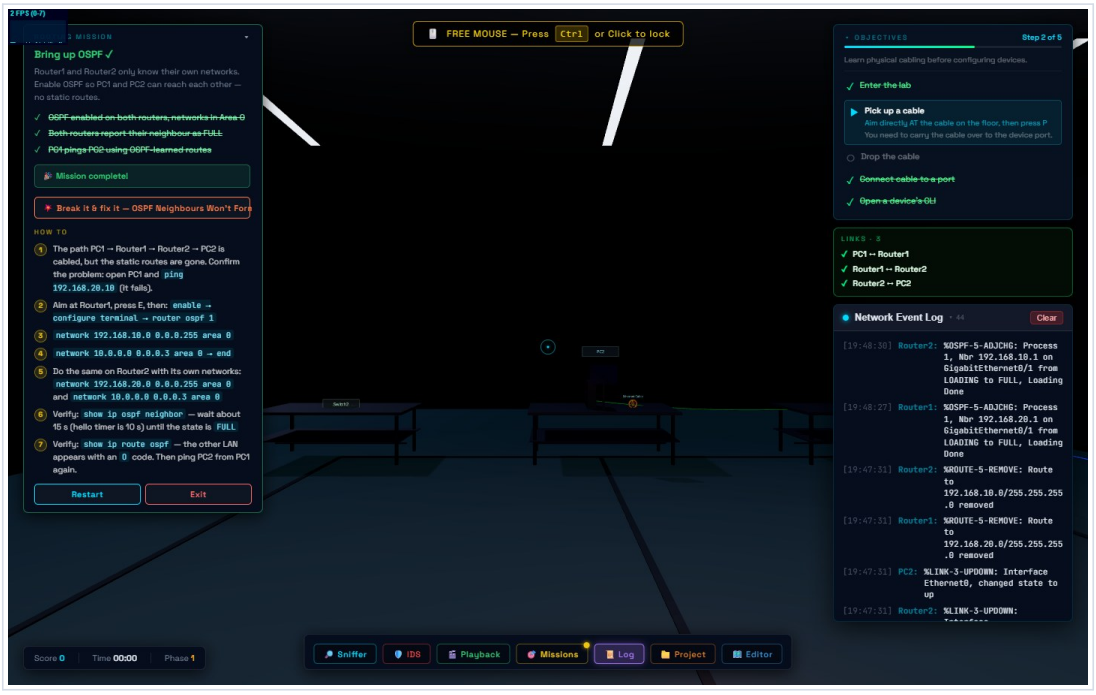


Figure 7 — The OSPF mission complete: all objectives ticked, and the follow-up lab offered.

Troubleshooting labs

Lab	The fault	How you find it
Wrong Subnet Mask	PC1 has a /26 mask, so its gateway looks off-subnet	`ip`, then `ping` the gateway
Shutdown Interface	Router1's port is administratively down	`show ip interface brief`
Wrong Access VLAN	PC1's switch port is in VLAN 20, PC2's in VLAN 10	`show vlan brief`
Missing Static Route	Router1 has no route to PC2's LAN	`show ip route`
OSPF Neighbours Won't Form	Router2 puts the link in Area 1	`show ip ospf neighbor` / `interface`

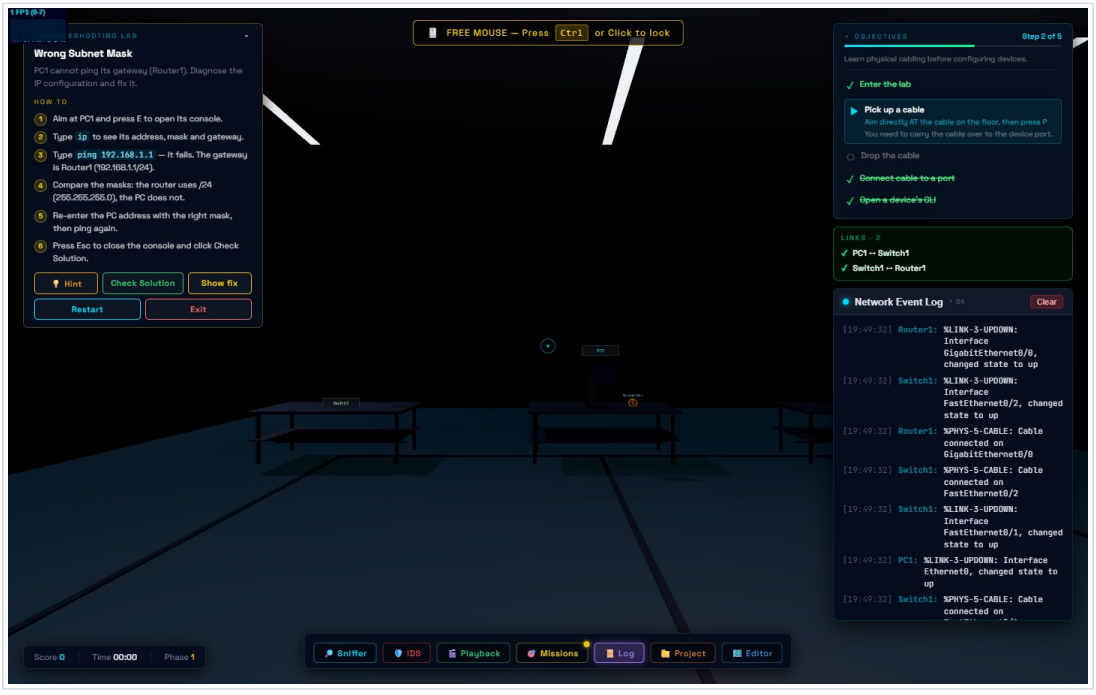


Figure 8 — A troubleshooting lab: walkthrough, Check Solution, Show fix, and live cable links.

08 Intrusion Detection System

The IDS window opens at the top of the screen so it stays off the console you are typing into, and it can be dragged. Alerts are kept while the window is closed; the toolbar badge counts the unread ones. Filter pills toggle each alert type and show a running count.

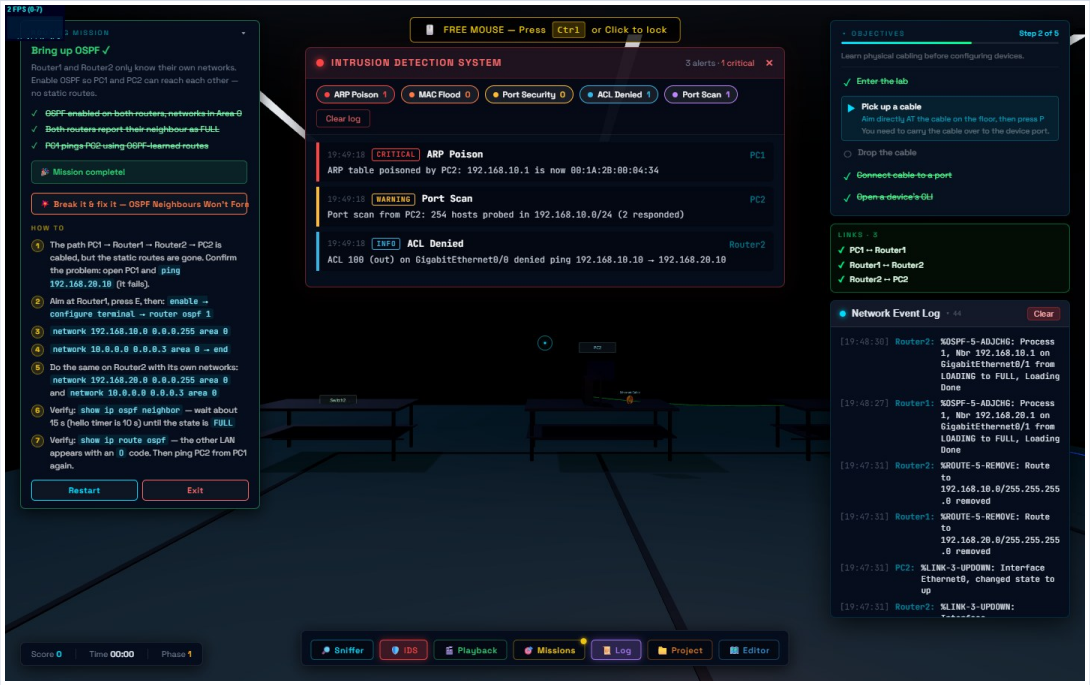


Figure 9 — Three alerts from one session: a critical ARP poisoning of PC1 by PC2, a port scan, and an ACL denial.

Alert	Severity	Raised when
ARP Poison	critical	A forged ARP reply changes a victim's ARP cache (`arpspoof`)
MAC Flood	critical	The switch's MAC address table overflows (8192 entries) and it starts flooding
Port Security	warning	An unauthorised MAC appears on a secured port
ACL Denied	info	An access list drops a packet (inbound or outbound)
Port Scan	warning	A sweep probes 8 or more hosts (`nmap`)

> **Honest detail:** the built-in `arpspoof` injects its forged reply straight into the victim's ARP cache, so Dynamic ARP Inspection cannot stop it in this simulator; the exercise is to *detect* it in the IDS. DAI/DHCP snooping can still be configured for practice.

09 The lab and its defaults

The lab boots **already configured**: every device has its address and the routers have static routes. Only the cabling is missing.

```
PC1 --- SW1 --- R1 --- R2 --- SW2 --- PC2
```

Device	Interface	Address
Router1	Gi0/0	192.168.10.1/24 (LAN toward PC1)

Device	Interface	Address
Router1	Gi0/1	10.0.0.1/30 (link to Router2)
Router2	Gi0/0	192.168.20.1/24 (LAN toward PC2)
Router2	Gi0/1	10.0.0.2/30 (link to Router1)
PC1	Eth0	192.168.10.10/24, gateway 192.168.10.1
PC2 (attacker)	Eth0	192.168.20.10/24, gateway 192.168.20.1

Static routes: Router1 reaches 192.168.20.0/24 via 10.0.0.2, and Router2 reaches 192.168.10.0/24 via 10.0.0.1. Switch ports start **administratively down** (unlike a real Catalyst), so a used switch port needs `no shutdown`.

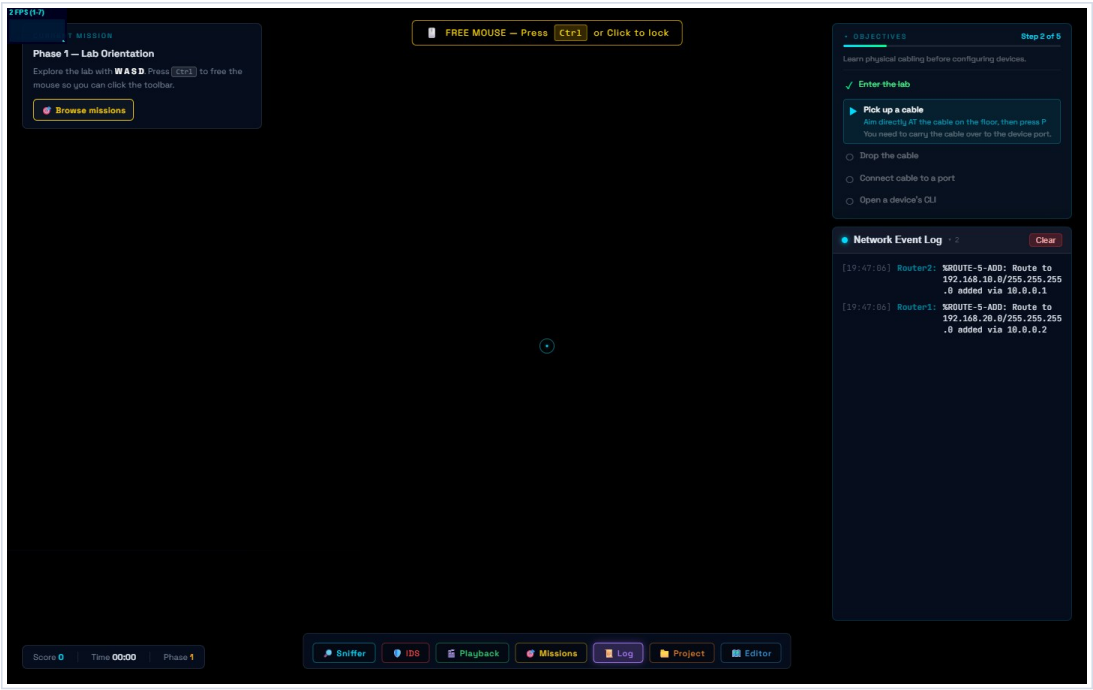


Figure 10 — The lab as it first appears, before any cabling. (Rendered with software graphics for this guide, so the room looks darker than on a GPU.)

10 Tutorial: from start to finish

Part 1 — Start the project

1. Open a terminal in the project folder and run `npm install` (first time only).
2. Run `npm run dev` and open `http://localhost:5173/` in the browser.
3. Click the screen to enter the lab. Walk with **W A S D**, look with the mouse. Press **Ctrl** whenever you need the cursor to click a button.

Part 2 — Wire the lab and make the first ping

1. Aim at a coiled cable on a desk and press **P**.
2. Aim at PC1's **Ethernet0** port until it glows and press **C**; then aim at Router1's **GigabitEthernet0/0** and press **C**.
3. Second cable: Router1 **Gi0/1** to Router2 **Gi0/1**. Third cable: Router2 **Gi0/0** to PC2 **Ethernet0**.
4. Aim at PC1 and press **E**. Type `ping 192.168.20.10`. Four replies mean packets crossed the whole lab.
5. Press **Esc** to leave. The Objectives panel ticks off: pick up a cable, drop it (G), connect it, open a CLI.

Part 3 — Missions (suggested order)

Press **Ctrl**, click **Missions**, choose one, and press **Hint** whenever you are unsure.

#	Mission	Do this
1	Catch the scanner	Open the IDS. On PC2: `nmap 192.168.10.0/24`
2	Detect the MITM	On PC2: `arpspoof --target 192.168.10.10 --gateway 192.168.10.1`
3	Lock the network down	Switch1: `enable`, `configure terminal`, then on interface Fa0/1 and Fa0/2: `switchport mode access`, `switchport port-security`. Then `macof` on PC2
4	Block traffic with an ACL	Ping PC2 from PC1. On Router2 add `access-list 100 deny icmp 192.168.10.0 0.0.0.255 192.168.20.0 0.0.0.255`, `access-list 100 permit ip any any`, then `ip access-group 100 out` on Gi0/0. Ping again
5	Bring up OSPF	On each router: `router ospf 1`, then two `network ... area 0` lines (the LAN and 10.0.0.0 0.0.0.3). Wait 15 s, `show ip ospf neighbor` until FULL, then ping

Part 4 — Troubleshooting labs and finishing

- Each lab is in the same Missions list. Follow the steps, fix the fault, then click **Check Solution**. **Show fix** reveals the answer if you are stuck.
- You are finished when every mission and lab in the list shows a tick: 4 security missions, the OSPF mission and 5 labs.
- **Reset lab to default** (bottom of the list) unplugs everything, restores the default addresses and clears all console history.

> **Tips.** Console history and device configs are saved automatically, so you can switch devices or refresh the page. After a refresh type `enable` again. Mission and lab ticks are *not* saved across a refresh. Type `hint` in any console for the next step.

11 Technology and project structure

Package	Purpose
React 19	User interface (HUD, panels, consoles)
Vite 8	Dev server and bundler
TypeScript 6	Type safety across engine and UI
Three.js r185 + @react-three/fiber 9 + drei 10	3D rendering, scene graph, helpers
react-draggable	Draggable windows such as the IDS
Vitest	Automated tests
oxlint	Linting

Source layout

Folder	Contents
`src/engine`	NetworkTopologyEngine: graph queries, reachability, VLAN-aware Layer-2 domain check
`src/network`	NetworkStore, SecurityEventBus, React hooks
`src/network/engine`	Devices, CLI engine, simulation engine, OSPF/RIP/DHCP/DNS/ACL/NAT, MissionEngine, ScenarioManager, ChallengeHints, LabReset, LabPersistence
`src/terminal`	TerminalSession (per-device consoles), key handling, ANSI colouring, banners
`src/scene`	Lab room, cables, MacBook console station, camera shots, packet animation
`src/player`	First-person look, physics and crosshair interaction
`src/components`	HUD rails, mission tracker/picker, IDS window, terminal UI, sniffer, playback, editor
`test-*.test.ts (root)`	Suites for missions, scenarios, OSPF and hints, persistence

Running it

```
npm install         # first time only
npm run dev         # http://localhost:5173/
npm run typecheck   # tsc -b --noEmit
npm test            # vitest, single run
npm run build       # production bundle in dist/
```

> **Safety.** This is a simulator. All commands are parsed and executed inside the browser against in-memory device objects. The app never uses `child_process`, PowerShell, CMD or any real shell.

12 Quality, limits and roadmap

Testing

- The suites cover all missions and labs end to end (including OSPF convergence with simulated time), hint progression, console records, and autosave round-trips of VLANs, ACLs, port security and cabling.
- Currently **86 tests pass**. Thirteen older suites under `src/network/engine/__tests__/` and three legacy tests fail; the older suites fail to import because of a circular import in the engine. That was already the case before the newest features and is tracked as a known issue.
- Type-checking reports no new errors from recent work; a set of older type warnings remains.

Known limits

- Mission and lab ticks, dropped-cable positions and topology-editor devices are not saved across a refresh.
- The engine clock follows real time, so OSPF takes about 15 seconds to converge.
- The optional GNS3 bridge is not connected; the app falls back to the built-in engine (and says so in the console).
- Scoring is not implemented yet; the Score / Time fields in the HUD are placeholders.

Roadmap

Phase	Status	Description
1-6	Done	3D lab, first-person player, cabling, topology engine, device models, Cisco-style CLI
7-10	Done	IP configuration, routing, ping, missions, OSPF
12-13	Done	VLAN / DHCP / NAT; security missions and the IDS
13b	Done	Per-device console history, lab autosave, mission hints, docked HUD
11	Partly	Troubleshooting labs and hints done; scoring to do
14-15	Planned	AI tutor (Gemini API through a FastAPI backend); backend API with SQLite for progress
16-17	Planned	Polish, testing and optimisation; optional GNS3 integration

A Appendix: keys and commands

Walking around the lab

Key	Action
W A S D / Mouse	Walk / look around
Shift / Space	Sprint (1.8x) / jump
E	Aim at a device and open its console
P / G	Pick up a cable or device / drop it
C	Connect the held cable to the port you aim at (creates a cable if you hold none)
X or D	Unplug the cable from a port
Ctrl	Free the mouse so you can click the HUD; press again to look around
Esc	Release the mouse / close the console

Cisco-style commands (routers and switches)

Area	Commands
Modes	enable, disable, configure terminal, exit, end, write memory
Show	show ip interface brief, show interface, show ip route [ospf], show ip ospf neighbor / interface / database, show mac address-table, show vlan brief, show port-security [interface X], show access-lists, show running-config, show version
Interfaces	interface X, ip address A M, no shutdown / shutdown, description
Routing	ip route N M NEXTHOP, router ospf 1, network N WILDCARD area 0, router rip, version 2, passive-interface
Switching	vlan N, switchport mode access|trunk, switchport access vlan N
Security	access-list, ip access-group X in|out, switchport port-security [maximum|violation], ip dhcp snooping, ip arp inspection trust, ip nat inside|outside

PC commands

Command	What it does
`ip A M [GW]`	Set address, mask and optional gateway (also brings the interface up)
`ip / show ip`	Show the current configuration
`ping H / traceroute H`	Test connectivity
`arp / route`	Show the ARP table / routing table
`save`	Save the PC configuration
`nmap CIDR`	Sweep a subnet for hosts and services (raises a Port Scan alert when it covers 8+ hosts)
`arpspoof --target T --gateway G`	Forge ARP replies (attacker PC2 only)
`macof [count]`	Flood the switch with random source MACs, default 10000 (attacker PC2 only)